

Execution Context

Execution Context is the **environment in which JavaScript code is evaluated and executed**.

Whenever JS runs a piece of code, it creates an execution context that contains:

- What variables exist
 - What functions are available
 - What this refers to
 - How the code should run
-



Think of It Like:

Execution Context = **A box where JS stores everything it needs to run your code.**



Types of Execution Context

1 Global Execution Context (GEC)

- Created when the program starts.
- Runs top-level code.
- this refers to the global object.
- Only **one** global context exists.

2 Function Execution Context (FEC)

- Created **every time a function is called**.
 - Each function gets its own separate environment.
-

Execution Context Has Two Phases

1 Creation Phase (Memory Setup)

JS scans code and:

- Allocates memory for variables → initialized as undefined
- Stores function definitions
- Sets up scope chain
- Decides this value

2 Execution Phase (Code Runs)

- Assigns real values to variables
 - Executes code line by line
-

Example

```
var x = 10;
```

```
function test() {  
  var y = 20;  
  return x + y;  
}
```

```
test();
```

Execution flow:

Global Execution Context created → x stored, test stored

test() called → New Function Execution Context created

y stored → code executed → result returned → context destroyed

Interview-Ready Answer

Execution Context is the environment JavaScript creates to run code. It contains variable declarations, function references, the scope chain, and the `this` binding. A global execution context is created first, and a new function execution context is created every time a function is invoked.

Call Stack

The **Call Stack** is a **data structure** that JavaScript uses to keep track of function execution — it follows the **Last In, First Out (LIFO)** principle.

👉 It records **which function is currently running and what should run next**.

Think of It Like a Stack of Plates

- The last plate placed on top is the first one removed.
 - Same with functions:
 - Last called → First finished.
-

How It Works

When JavaScript runs code:

- 1 The **Global Execution Context** is pushed onto the stack.
 - 2 When a function is called → its execution context is **pushed** onto the stack.
 - 3 When the function finishes → it is **popped** off the stack.
 - 4 Control returns to the previous function.
-

Example

```
function one() {  
  two();  
}
```

```
}
```

```
function two() {  
  console.log("Hello");  
}
```

```
one();
```

Stack Flow:

Start Program

↓

Push Global()

↓

Push one()

↓

Push two()

↓

Execute console.log

↓

Pop two()

↓

Pop one()

↓

Back to Global()

↓

End

Why Call Stack Is Important

Concept	Role of Call Stack
Execution Order	Ensures functions run in sequence
Error Tracking	Stack traces come from here
Single Thread	JS can run only one task at a time
Recursion Control	Prevents infinite calls (Stack Overflow)

Common Interview Point

JavaScript is **single-threaded** because it has **one call stack** — it can execute only one function at a time.

Async behavior happens **outside** the stack (via Web APIs & Event Loop).

Interview-Ready Answer

The Call Stack is a LIFO data structure used by JavaScript to manage execution contexts. Each function call is pushed onto the stack, and when it completes, it is popped off, ensuring code executes in the correct order.